

API GA7-220501096-AA5-EV02

JOSE ALEXANDER ORTIZ RUBIO

Ficha 3336243

Centro Ambiental y Ecoturístico del Nororiente Amazónico

SENA Regional Guainía

Tecnólogo en Análisis y Desarrollo de Software-ADSO

Instructora Yinna Paola Varón Moreira

Bogotá, Colombia

31 de Agosto de 2026

Contenido

Codigo.....	3
Evidencias.....	10
Registrar Usuario	10
Error	10
Usuario Registrado	11
Consultar Usuarios.....	14
Eliminar 1 usuario.....	15
Error	16
Usuario eliminado	17
Eliminar Todos los usuarios	18
Login.....	19
Error	21
Login Exitoso	22

Codigo

```
const express = require("express");

const app = express();

// Permite recibir información en formato JSON
app.use(express.json());

/*
Tecnologo en Analisis y Desarrollo de Software
GA7-220501096-AA5-EV02
Jose Alexander Ortiz Rubio
*/

const usuarios = [];

// =====

// RUTA PRINCIPAL

// =====

app.get("/", (req, res) => {
    res.send("API de autenticación funcionando correctamente");
});
```

```
// =====

// SERVICIO DE REGISTRO

// =====

app.post("/api/autenticar/registro", (req, res) => {

  // Obtener usuario y contraseña enviados

  const { usuario, password } = req.body;

  // Validar que los datos sean obligatorios

  if (!usuario || !password) {

    return res.status(400).json({

      error: "¡Atención! El usuario y la contraseña son obligatorios"

    });

  }

  // Verificar si el usuario ya existe

  const usuarioExiste = usuarios.find(

    u => u.usuario === usuario

  );

  if (usuarioExiste) {
```

```
        return res.status(409).json({
            error: "Usuario ya registrado"
        });
    }

    // Registrar el nuevo usuario
    usuarios.push({
        usuario: usuario,
        password: password
    });

    // Respuesta del servicio
    res.status(201).json({
        mensaje: "Usuario registrado exitosamente"
    });
});

// =====

// SERVICIO PARA LISTAR USUARIOS

// =====

app.get("/api/consulta/usuarios", (req, res) => {
```

```

// Devolver la lista de usuarios registrados

res.status(200).json(usuarios);

});

// =====

// SERVICIO PARA ELIMINAR UN USUARIO

// =====

app.delete("/api/autenticar/usuarios/:usuario", (req, res) => {

// Obtener el nombre de usuario enviado en la URL

const usuario = req.params.usuario;

// Buscar la posición del usuario en el arreglo

const indice = usuarios.findIndex(

  u => u.usuario === usuario

);

if (indice === -1) {

  return res.status(404).json({

    error: "El usuario no existe"

  });

}

```

```
// Eliminar el usuario del arreglo

usuarios.splice(indice, 1);


// Informar que la eliminación fue exitosa

res.status(200).json({

  mensaje: "El usuario ha sido eliminado correctamente"

});

});


// ELIMINAR TODOS LOS USUARIOS


app.delete("/api/autenticar/usuarios", (req, res) => {


  // Eliminar todos los usuarios

  usuarios.length = 0;


  res.status(200).json({

    mensaje: "Todos los usuarios han sido eliminados correctamente"

  });

});


// =====
```

```
// SERVICIO DE INICIO DE SESIÓN

// =====

app.post("/api/autenticar/login", (req, res) => {

  // Obtener las credenciales enviadas

  const { usuario, password } = req.body;

  // Validar que los datos sean obligatorios

  if (!usuario || !password) {

    return res.status(400).json({

      error: "El usuario y la contraseña son obligatorios"

    });

  }

  // Buscar usuario y comprobar la contraseña

  const usuarioEncontrado = usuarios.find(

    u => u.usuario === usuario &&

    u.password === password

  );

  // Si las credenciales son incorrectas

  if (!usuarioEncontrado) {
```

```
        return res.status(401).json({
            error: "Error en autenticación"
        });
    }

    // Si las credenciales son correctas
    res.status(200).json({
        mensaje: "Autenticación satisfactoria"
    });
});

// =====

// INICIAR SERVIDOR

// =====

const PORT = 3000;

app.listen(PORT, () => {
    console.log(`Servidor ejecutándose en http://localhost:${PORT}`);
});
```

Evidencias

Registrar Usuario

POST -> http://localhost:3000/api/autenticar/registro

Error

The screenshot displays a REST client interface with the following details:

- Request:** POST to `http://localhost:3000/api/autenticar/registro`. The body is a JSON object:

```
{ 1: { 2: "usuario": "Jose", 3: "password": "", 4: } 5: }
```

.
- Response:** 400 Bad Request (3 ms, 315 B). The JSON body is:

```
{ 1: { 2: "error": "¡Atención! El usuario y la contraseña son obligatorios" 3: }
```

.
- AI Chat:** A sidebar on the right titled "New Chat" provides a suggestion: "Add missing password. Provide both the username and password in the request body." It shows a diff view where the password field is added with the value "ABC123".

Usuario Registrado

The screenshot displays a REST client interface with the following details:

- Request:** A POST request to `http://localhost:3000/api/autenticar/registro`. The body is a JSON object:

```
{  "usuario": "Jose",  "password": "ABC123"}
```

.
- Response:** A 201 Created status with a response body:

```
{  "mensaje": "Usuario registrado exitosamente"}
```

.
- Metadata:** The response is 25 ms and 287 B in size.
- UI Elements:** The interface includes a sidebar with 'COLLECTIONS' and 'ENVIRONMENTS', a top bar with search and navigation, and a bottom status bar indicating 'No anomalies detected'.

The screenshot displays the Postman application interface. At the top, the workspace is named "Alexander DesktopManagement's Workspace". The main area shows a POST request to the URL `http://localhost:3000/api/autenticar/registro`. The request body is a JSON object:

```
1 {
2   "usuario": "Matias",
3   "password": "DEF456"
4 }
```

The response is shown below the request, indicating a `201 Created` status with a response time of 3 ms and a body size of 287 B. The response body is a JSON object:

```
1 {
2   "mensaje": "Usuario registrado exitosamente"
3 }
```

On the left sidebar, the "COLLECTIONS" section is expanded, showing "My Collection". The bottom status bar indicates "No anomalies detected". On the right, there is a "New Chat" button and a chat interface with a prompt: "Describe what you need. Press @ for context, / for Skills." and an "Auto" dropdown menu.

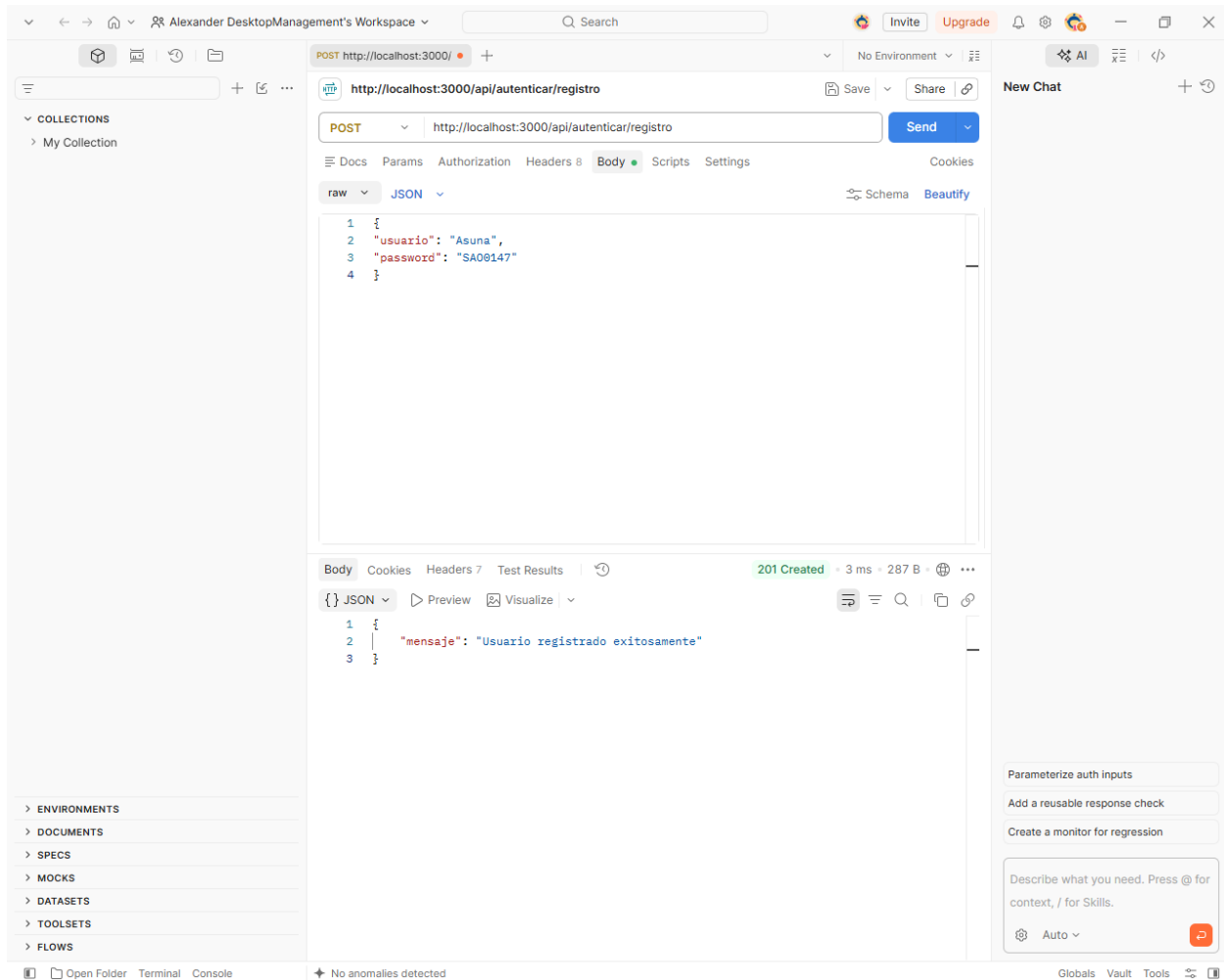
The screenshot displays the Postman application interface. The top bar shows the workspace name "Alexander DesktopManagement's Workspace" and a search bar. The left sidebar contains a "COLLECTIONS" section with "My Collection" and an "ENVIRONMENTS" section with various categories like DOCUMENTS, SPECS, MOCKS, DATASETS, TOOLSETS, and FLOWS. The main workspace is configured for a POST request to the URL "http://localhost:3000/api/autenticar/registro". The request body is a JSON object:

```
{  "usuario": "Samy",  "password": "GHI789"}
```

. The response is displayed in the "Body" tab, showing a 201 status code with the message:

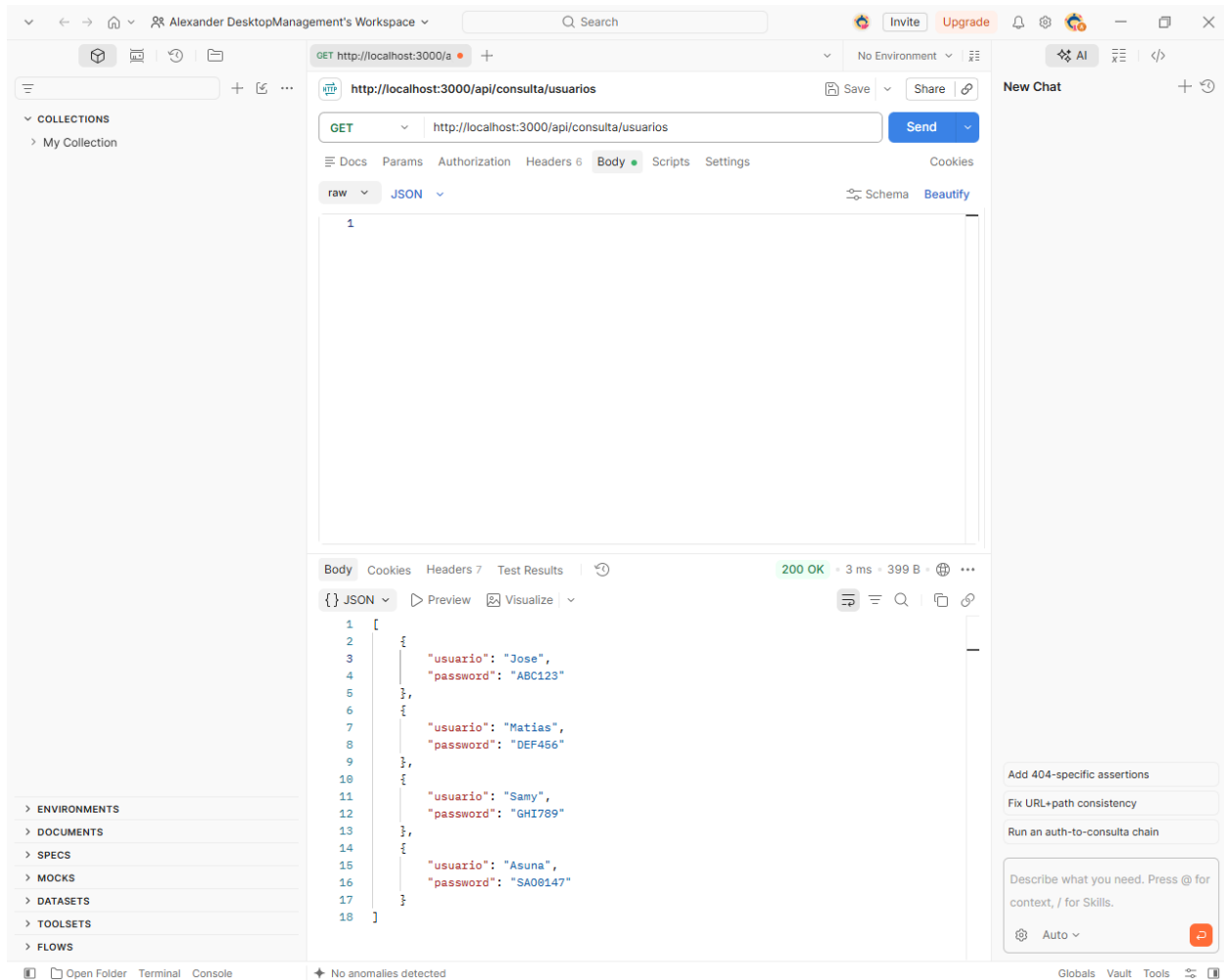
```
{  "mensaje": "Usuario registrado exitosamente"}
```

. The status bar at the bottom indicates "No anomalies detected". On the right side, there is a "New Chat" button and a chat interface with a prompt: "Describe what you need. Press @ for context, / for Skills." and an "Auto" dropdown menu.



Consultar Usuarios

GET -> <http://localhost:3000/api/consulta/usuarios>



Eliminar 1 usuario

DELETE -> <http://localhost:3000/api/autenticar/usuarios/:usuario>

Error

The screenshot displays the Postman interface with a workspace named "Alexander DesktopManagement's Workspace". The active request is a **DELETE** method to the URL `http://localhost:3000/api/autenticar/usuarios/jose`. The request body is set to `none`. The response status is **404 Not Found**, with a response time of 2 ms and a size of 276 B. The response body is in JSON format, showing an error message: `{ "error": "El usuario no existe" }`. The interface includes a sidebar with "COLLECTIONS" (My Collection) and "ENVIRONMENTS" (DOCUMENTS, SPECS, MOCKS, DATASETS, TOOLSETS, FLOWS). The bottom status bar shows the error message "Investigate user not found error".

Request Details:

- Method: DELETE
- URL: `http://localhost:3000/api/autenticar/usuarios/jose`
- Body: none

Response Details:

- Status: 404 Not Found
- Time: 2 ms
- Size: 276 B
- Body (JSON):

```
{  "error": "El usuario no existe"}
```

AI Chat Panel:

New Chat

- Investigate user not found error
 - Continue in Agent Mode to investigate why the user resource does not exist.
 - Open Source Code Think Harder

Footer:

- Open Folder Terminal Console
- Investigate user not found error
- Globals Vault Tools

Usuario eliminado

The screenshot displays a REST client interface with the following components:

- Top Bar:** Includes navigation icons, a search bar, and buttons for 'Invite' and 'Upgrade'.
- Left Panel:** Shows a sidebar with 'COLLECTIONS' and 'My Collection'.
- Request Section:**
 - Method:** DELETE
 - URL:** http://localhost:3000/api/autenticar/usuarios/Jose
 - Body:** none
- Response Section:**
 - Status:** 200 OK
 - Time:** 2 ms
 - Size:** 293 B
 - Body:** A JSON object:

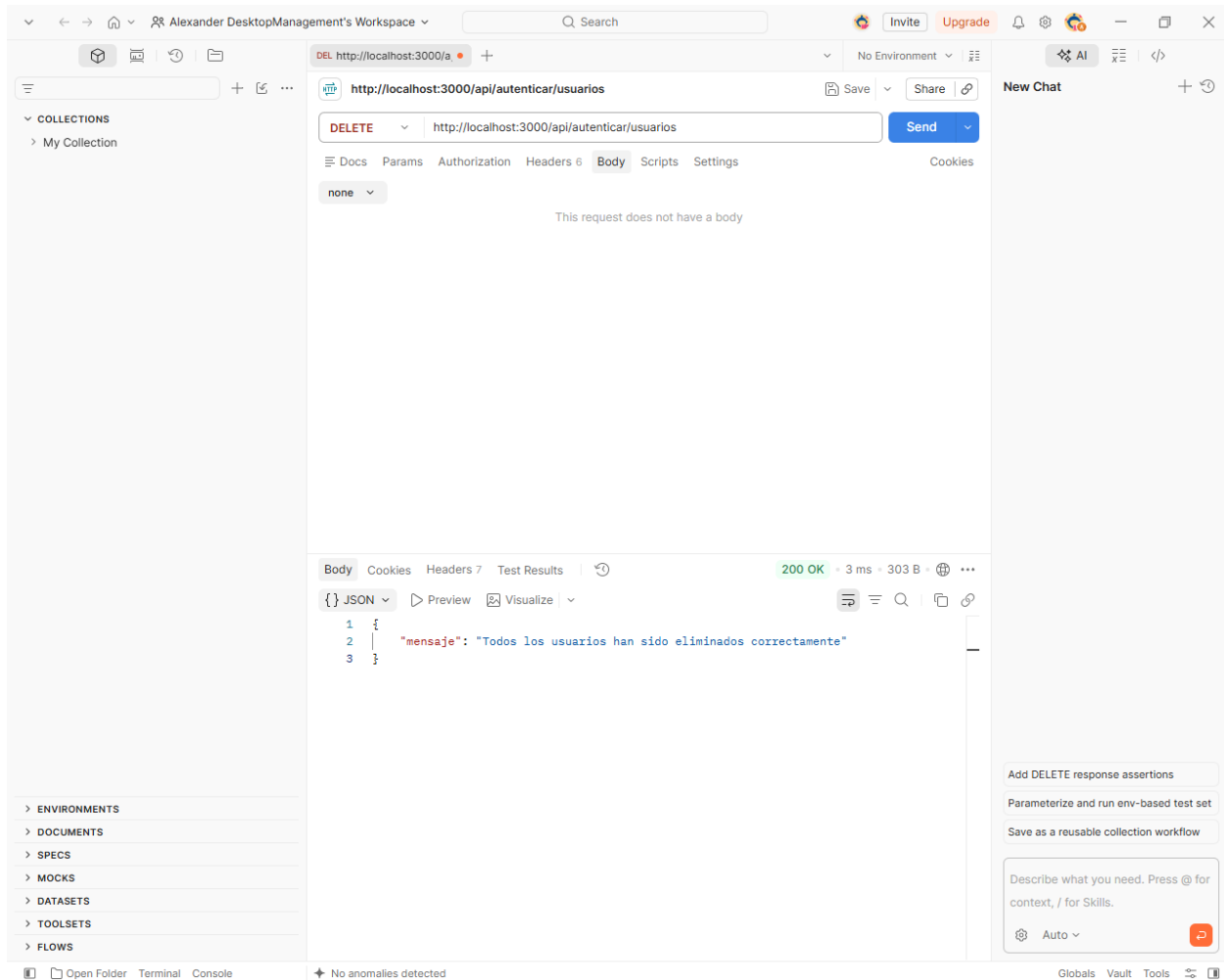
```
{  "mensaje": "El usuario ha sido eliminado correctamente"}
```
- Bottom Panel:** Includes a terminal/console area with the message 'No anomalies detected'.
- Right Panel:** Contains a 'New Chat' section with a chat interface and a 'Fix base path & add guard tests' button.

The screenshot displays the Postman application interface. At the top, the workspace is named "Alexander DesktopManagement's Workspace". The main area shows a GET request to the endpoint `http://localhost:3000/api/consulta/usuarios`. The request body is set to "none". Below the request, the response is shown as a 200 OK status with a response time of 2 ms and a body size of 360 B. The response body is a JSON array of three user objects, displayed in a syntax-highlighted format with line numbers 1 through 14. The left sidebar contains a "COLLECTIONS" section with "My Collection" and an "ENVIRONMENTS" section. The bottom status bar indicates "No anomalies detected". On the right, there is a "New Chat" button and a chat interface with a text input and a "Send" button.

```
1 [
2   {
3     "usuario": "Matias",
4     "password": "DEF456"
5   },
6   {
7     "usuario": "Samy",
8     "password": "GHI789"
9   },
10  {
11    "usuario": "Asuna",
12    "password": "SA08147"
13  }
14 ]
```

Eliminar Todos los usuarios

DELETE -> <http://localhost:3000/api/autenticar/usuarios>



Login

POST -> <http://localhost:3000/api/autenticar/login>

The screenshot displays the Postman application interface. At the top, the workspace is named "Alexander DesktopManagement's Workspace". The main area shows a GET request to the URL `http://localhost:3000/api/consulta/usuarios`. The request body is set to "none", with a note stating "This request does not have a body". The response is a 200 OK status, received in 3 ms with a body size of 277 B. The response body is displayed in JSON format, showing an array of one object with the following structure:

```
1 [
2   {
3     "usuario": "Jose",
4     "password": "ABC123"
5   }
6 ]
```

The left sidebar contains a "COLLECTIONS" section with "My Collection" and an "ENVIRONMENTS" section. The bottom status bar indicates "No anomalies detected". On the right, there is a "New Chat" section with a text input and a "Send" button, and a "Rebuild registro request body" button.

Error

The screenshot shows the Postman interface with a REST client request to `http://localhost:3000/api/autenticar/login` using the POST method. The request body is a JSON object: `{ "usuario": "Matias", "password": "DEF456" }`. The response is a 401 Unauthorized status with a response time of 3 ms and a body size of 282 B. The response body is a JSON object: `{ "error": "Error en autenticación" }`. The interface includes a sidebar with collections and environments, a top bar with search and navigation, and a bottom bar with terminal and console options. A 'New Chat' panel is visible on the right side.

POST `http://localhost:3000/api/autenticar/login`

Body

```
1 {
2   "usuario": "Matias",
3   "password": "DEF456"
4 }
```

401 Unauthorized · 3 ms · 282 B

Troubleshoot authentication failure

Fix auth Decline

JSON Preview Pass the correct auth credentials

```
1 {
2   "error": "Error en autenticación"
3 }
```

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLOWS

Open Folder Terminal Console

Troubleshoot authentication failure

Globals Vault Tools

New Chat

Troubleshoot authentication failure

Continue in Agent Mode to troubleshoot the authentication failure.

Open Source Code Think Harder

Create tests for POST validation

Parameterize POST payload

Add a fallback POST/GET parity check

Describe what you need. Press @ for context, / for Skills.

Auto

Login Exitoso

The screenshot displays the Postman application interface. The top bar shows the workspace name "Alexander DesktopManagement's Workspace" and a search bar. The left sidebar contains a "COLLECTIONS" section with "My Collection" and an "ENVIRONMENTS" section with various categories like DOCUMENTS, SPECS, MOCKS, DATASETS, TOOLSETS, and FLOWS. The main workspace is set to "No Environment". The request is a POST to "http://localhost:3000/api/autenticar/login". The request body is a JSON object:

```
{  "usuario": "Jose",  "password": "ABC123"}
```

. The response is a 200 OK status with a response time of 3 ms and a body size of 279 B. The response body is a JSON object:

```
{  "mensaje": "Autenticación satisfactoria"}
```

. The bottom status bar indicates "No anomalies detected". On the right side, there is a "New Chat" button and a chat interface with a prompt "Describe what you need. Press @ for context, / for Skills." and an "Auto" dropdown menu.

POST http://localhost:3000/api/autenticar/login

Send

raw JSON

```
1 {
2   "usuario": "Jose",
3   "password": "ABC123"
4 }
```

Body Cookies Headers 7 Test Results

200 OK • 3 ms • 279 B

{ } JSON Preview Visualize

```
1 {
2   "mensaje": "Autenticación satisfactoria"
3 }
```

Open Folder Terminal Console

No anomalies detected

Globals Vault Tools

Add auth headers & token flow

Record expected 401 response tests

Create an environment for credentials

Describe what you need. Press @ for context, / for Skills.

Auto